

DESENVOLVIMENTO DE UMA APLICAÇÃO WEB PARA GESTÃO E ENCURTAMENTO DE LINKS SOB A ÓTICA DA ARQUITETURA MONOREPO

Acadêmico: Pedro Ryan Aranha da Costa

Turma: FLD6786572CET

Tutor: Cristiano Flores dos Santos

RESUMO

O compartilhamento de links curtos tornou-se essencial para profissionais e criadores de conteúdo na internet. No entanto, esses usuários frequentemente enfrentam obstáculos ao utilizar URLs longas, que prejudicam a usabilidade e a estética da comunicação. O uso de plataformas de terceiros para o encurtamento de links, embora prático, impõe limitações sobre o controle de dados, personalização e expiração das rotas. Visando solucionar essa dependência, esta pesquisa aplicada descreve o desenvolvimento de uma aplicação web de uso pessoal, denominada "EversonLinks", voltada ao gerenciamento e rastreamento de URLs. O sistema foi projetado sob uma arquitetura de repositório único (Monorepo), utilizando o runtime Bun e o framework ElysiaJS no back-end para garantir baixíssima latência e alta performance em operações de redirecionamento HTTP (Status 302). A persistência de dados foi implementada com SQLite e Prisma ORM, garantindo integridade e facilidade de migração futura. A interface de usuário foi construída com React, Vite e Shadcn UI, adotando o TanStack Query para um gerenciamento eficiente da separação entre o estado do cliente e do servidor. Os resultados obtidos resultaram em um Produto Mínimo Viável (MVP) totalmente funcional, contemplando operações de criação, leitura, atualização e exclusão (CRUD), além de métricas de cliques. Conclui-se que o uso de stacks tecnológicas modernas não apenas acelera o desenvolvimento de utilitários web, mas também permite a construção de infraestruturas independentes, escaláveis e com excelente experiência de usuário e desenvolvedor.

Palavras-chave: Encurtador de Links. Arquitetura Monorepo. Desenvolvimento Web.

INTRODUÇÃO

Para profissionais que dependem da internet como principal meio de comunicação e trabalho, como educadores e criadores de conteúdo, o uso de links curtos é essencial. Contudo, URLs (Uniform Resource Locators) extensas e complexas frequentemente dificultam a legibilidade, prejudicam a estética em materiais de divulgação e oferecem uma experiência de usuário (UX) sub-ideal. Embora existam diversas plataformas comerciais de encurtamento de links, essas soluções de terceiros

frequentemente impõem limitações em suas versões gratuitas, como a expiração de links, restrições na personalização e a retenção do controle dos dados de tráfego, criando uma dependência tecnológica indesejada para o profissional.

Neste cenário, a pergunta que moveu esta pesquisa foi: **Como desenvolver uma solução web independente, escalável e de alto desempenho para o encurtamento e a gestão de links, que garanta ao usuário a propriedade total de seus dados e métricas de acesso?**

Para responder a este questionamento, o **objetivo geral** deste trabalho é apresentar o desenvolvimento e a implementação de uma aplicação web de uso pessoal (single-tenant), denominada "EversonLinks", destinada ao gerenciamento, encurtamento e rastreamento básico de URLs.

Para o alcance do objetivo geral, foram traçados os seguintes objetivos específicos:

- Arquitetar o sistema utilizando o padrão *Monorepo*, separando as responsabilidades de cliente (Front-end) e servidor (Back-end) para facilitar a manutenção e escalabilidade;
- Desenvolver uma API RESTful de alta performance utilizando o runtime Bun e o framework ElysiaJS, garantindo respostas rápidas para os redirecionamentos;
- Implementar a persistência de dados e o registro de métricas (cliques) por meio de um banco de dados relacional (SQLite) orquestrado pelo Prisma ORM;
- Construir uma interface de usuário responsiva e acessível utilizando React, Vite e componentes Radix (Shadcn UI), aplicando boas práticas de separação entre estados de servidor (via React Query) e estados de cliente;
- Implementar as operações fundamentais de um sistema (CRUD: Criação, Leitura, Atualização e Exclusão), acompanhadas da lógica de redirecionamento HTTP nativo (Status 302).

FUNDAMENTAÇÃO TEÓRICA

O desenvolvimento de aplicações web modernas exige arquiteturas que favoreçam a manutenibilidade, a velocidade de execução e a separação de interesses. Segundo Sommerville (2011), estruturar um sistema em componentes e camadas independentes reduz a complexidade geral e facilita a evolução contínua do software. Nesse contexto, o presente projeto adotou a estrutura de Monorepo (repositório único). Essa abordagem permite centralizar os pacotes de front-end, back-end e regras de negócio de banco de dados num mesmo local, compartilhando configurações e dependências, ao mesmo tempo em que preserva o isolamento de suas

responsabilidades, inspirando-se nos princípios do padrão MVC (Model-View-Controller).

Para a camada de servidor (API RESTful), a escolha do ambiente de execução é crucial para a performance, especialmente em sistemas de encurtamento de links que demandam latência mínima no redirecionamento. De acordo com Pressman e Maxim (2016, p. 350), "o tempo de resposta de uma aplicação web é um fator determinante para a retenção do usuário". Por este motivo, optou-se pela utilização do runtime Bun em conjunto com o framework ElysiaJS. Estas tecnologias processam requisições HTTP de forma altamente otimizada e executam TypeScript de forma nativa, dispensando etapas custosas de compilação (build steps) e proporcionando uma excelente Developer Experience (DX).

Para garantir a segurança da API, aplicou-se a validação estrita de dados de entrada e saída utilizando esquemas tipados (schemas). O principal desafio técnico desta camada consistia em interceptar o acesso a um link curto, registrar a métrica de acesso e encaminhar o usuário corretamente. O Trecho de Código 1 demonstra a rota responsável por este comportamento.

Figura 1 – Trecho de código-fonte da rota pública de redirecionamento (ElysiaJS).

```
1 import Elysia from "elysia"
2 import { LinkModels } from "../models/links.models"
3 import { LinksService } from "../services/links.service"
4
5 export const PublicRoutes = new Elysia()
6   .use(LinkModels)
7   .get("/s/:shortCode",
8     async ({ params, status, redirect }) => {
9       const { shortCode } = params
10       const link = await LinksService.getLink(shortCode)
11
12       if (!link) return status(404)
13
14       return redirect(link.originalUrl, 302)
15     },
16     {
17       params: "links.params.shortCode",
18     }
19 )
20
```

Fonte: O autor (2026).

Conforme demonstrado na Figura 1, a aplicação utiliza explicitamente o código de status HTTP 302 (*Found* ou *Temporary Redirect*). Esta decisão arquitetural é um pilar técnico do projeto. Caso fosse utilizado o redirecionamento permanente (HTTP 301), os navegadores modernos armazenariam o destino em *cache* local. Consequentemente, acessos subsequentes não atingiriam o servidor, inviabilizando o rastreamento das métricas. O uso do HTTP 302 garante que toda requisição alcance a aplicação, permitindo o correto incremento da contagem de cliques no banco de dados.

Para a camada de persistência de dados, a escolha do Sistema Gerenciador de Banco de Dados (SGBD) e das ferramentas de integração deve alinhar-se à proposta de agilidade de um Produto Mínimo Viável (MVP). Elmasri e Navathe (2011) destacam que o modelo de dados relacional provê uma base sólida para a integridade, consistência e padronização das informações. Com base nisso, adotou-se o SQLite como banco de dados de desenvolvimento devido à sua eficiência, operando em conjunto com o *Object-Relational Mapper* (ORM) Prisma. A utilização do ORM atua como uma poderosa camada de abstração que blinda a aplicação contra dependências estritas do SGBD, facilitando migrações futuras para sistemas mais robustos e voltados para *analytics*, como o PostgreSQL, com o mínimo de refatoração no código.

A modelagem de dados do sistema "EversonLinks" foi projetada para ser estritamente funcional, focada em resolver o problema central sem gerar sobrecarga de informações. O Trecho de Código 2 apresenta a estrutura da tabela principal da aplicação, definida através da linguagem declarativa característica do Prisma.

Figura 2 – Esquema de modelagem da entidade Link utilizando Prisma ORM.

```
1 //...
2 model Link {
3   id          String    @id @default(cuid())
4   originalUrl String
5   shortCode   String    @unique
6   clicks      Int        @default(0)
7   createdAt   DateTime  @default(now())
8 }
9
```

Fonte: O autor (2026).

Observa-se, na Figura 2, a presença de atributos fundamentais para a regra de negócio. Destacam-se o campo de destino (**originalUrl**), o identificador único gerado aleatoriamente (**shortCode**) — cujo atributo **@unique** previne colisões e garante a integridade da busca — e o contador inteiro (**clicks**), que viabiliza o rastreamento das métricas.

Além da estruturação dos dados, a comunicação ininterrupta e fluida entre o cliente e a API representa um desafio arquitetural constante em aplicações *Single Page Application* (SPA). Para lidar com as limitações de segurança dos navegadores, o projeto implementou o padrão de Proxy Reverso através da configuração do servidor Vite. Conforme explicam Kurose e Ross (2013), a introdução de um *proxy* possibilita interceptar e tratar requisições de forma transparente. Na prática desta aplicação, isso não apenas soluciona restrições de segurança do navegador relacionadas ao *Cross-Origin Resource Sharing* (CORS) durante o desenvolvimento, mas também simula fidedignamente o comportamento de infraestruturas de produção, garantindo que a interface gráfica e o redirecionamento (via rotas */s/*) coexistam perfeitamente sob o mesmo domínio.

Ainda no contexto do servidor, a aplicação adotou o princípio arquitetural da Separação de Preocupações (*Separation of Concerns*). Em vez de acoplar a manipulação do banco de dados diretamente aos controladores das rotas, toda a regra de negócio foi isolada em uma camada de Serviços (*Services*). Esta abordagem garante que as regras de criação, atualização e listagem de dados sejam facilmente testáveis e reutilizáveis. O Trecho de código da Figura 3 ilustra a implementação desta camada.

Figura 3 – Classe de serviço responsável pela regra de negócio e geração do código curto.

```
1 export abstract class LinksService {
2   static async createLink(originalUrl: string): Promise<LinksModel.Link> {
3     if (!isValidUrl(originalUrl)) {
4       throw status(400, "Invalid URL format")
5     }
6
7     const shortCode = nanoid(6)
8
9     const link = await prisma.link.create({
10      data: {
11        originalUrl,
12        shortCode,
13      },
14    })
15
16    return link
17  }
18  // ... Listlinks, getLink, deleteLink, updateLink ...
19 }
20
```

Fonte: O autor (2026).

Conforme exposto na Figura 3, o método responsável pela criação do link curto realiza verificações prévias de formatação de URL e utiliza a biblioteca *nanoid* para gerar um identificador alfanumérico seguro e enxuto. Somente após estas validações a transação é enviada ao Prisma ORM. Adicionalmente, a consistência das requisições recebidas pela API é garantida através de esquemas de validação (*schemas*) declarados estaticamente utilizando a tipagem avançada do Elysia (via TypeBox). Isso significa que qualquer dado malformatado é rejeitado na entrada da requisição, poupando recursos de processamento e evitando anomalias no banco de dados.

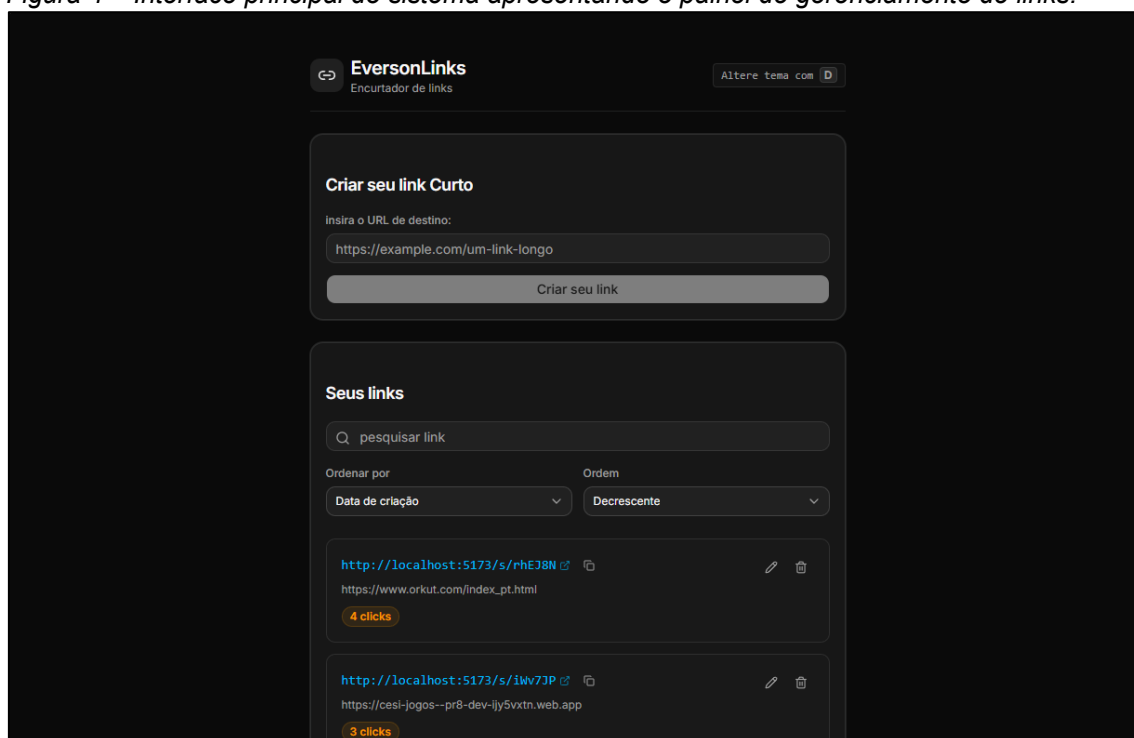
Para a Interface de Usuário (UI), optou-se pela biblioteca React, muito utilizada na construção de Single Page Applications (SPAs). A transição para esse modelo arquitetural transfere grande parte do processamento e controle de estado para o navegador do cliente, exigindo interfaces altamente reativas. Sobre a complexidade e os requisitos de aplicações *web* modernas, Pressman e Maxim (2016, p. 382) destacam:

As interfaces de usuário para WebApps enfrentam desafios únicos, exigindo projetos que acomodem uma ampla variedade de dispositivos e garantam extrema fluidez. Uma interface bem projetada deve proporcionar interação imediata e feedback visual contínuo, minimizando o tempo de espera percebido. Isso é alcançado transferindo a lógica de estado e processamento para o cliente, permitindo que a aplicação atualize dinamicamente partes específicas da tela sem a necessidade de solicitar o recarregamento da página inteira ao servidor, melhorando substancialmente a experiência do usuário.

Para solucionar o desafio de gerenciamento de estado mencionado pelos autores, optou-se pela separação conceitual entre o Estado de Interface (*Client State*) e o Estado de Servidor (*Server State*). Utilizou-se a biblioteca TanStack Query (React Query) para lidar exclusivamente com o *cache*, sincronização, estados de carregamento (*loading*) e tratativas de erro das requisições HTTP, evitando a criação de lógicas repetitivas e garantindo que a lista de links na tela esteja sempre sincronizada com o banco de dados de forma assíncrona.

A Figura 4 apresenta a interface principal gerada por essa arquitetura.

Figura 4 – Interface principal do sistema apresentando o painel de gerenciamento de links.



Fonte: O autor (2026).

O *design* da interface (Figura 4) foi projetado sob o paradigma *Developer-First*, adotando um tema escuro e minimalista. Para agilizar o desenvolvimento visual sem perder a performance, utilizou-se o framework TailwindCSS, integrado a componentes modulares e estilizados do ecossistema Shadcn UI. Essa combinação permitiu a entrega de componentes complexos — como modais de exclusão, paginação, formulários reativos e alertas visuais (com *feedback* instantâneo de sucesso ou erro) — mantendo o foco total na usabilidade (UX) e na rapidez operacional exigida pelo usuário final.

METODOLOGIA

A presente pesquisa classifica-se, quanto à sua natureza, como **aplicada**, pois buscou gerar conhecimentos tecnológicos para a solução de um problema prático e

específico: a gestão e o encurtamento de URLs. Quanto à abordagem, trata-se de uma pesquisa **qualitativa**, uma vez que o estudo não se deteve em levantamentos estatísticos ou matemáticos, mas sim na compreensão profunda de conceitos arquiteturais e na avaliação da qualidade e eficiência das ferramentas de desenvolvimento adotadas.

Quanto ao tipo de pesquisa, o trabalho configura-se como **bibliográfico e documental**. As **fontes de informações** utilizadas para embasar as decisões técnicas foram livros acadêmicos consolidados na área de Engenharia de Software (como Pressman e Sommerville), bem como a leitura analítica das documentações técnicas oficiais (*web*) das tecnologias implementadas (como Bun, React e Prisma).

Devido à natureza prática e tecnológica deste projeto, os **instrumentais de pesquisa** tradicionais, como questionários ou formulários com pessoas, não se aplicavam. Em seu lugar, utilizou-se como método de coleta e validação de dados a **prototipação e experimentação**. O principal "instrumento" utilizado para se chegar aos resultados foi o desenvolvimento iterativo de um Produto Mínimo Viável (MVP) em ambiente local, programado através da IDE (*Integrated Development Environment*) Visual Studio Code.

O caminho percorrido até os resultados finais ocorreu em etapas: inicialmente, configurou-se uma arquitetura de repositório unificado (*Monorepo*); em seguida, desenvolveu-se a API (*back-end*) com ElysiaJS e o banco de dados (SQLite); por fim, construiu-se a interface (*front-end*) com React e Tailwind. Os resultados e o comportamento do sistema foram testados e validados empiricamente pelo próprio autor através de requisições no navegador. Para garantir a transparência do processo e permitir a avaliação da prova de conceito, todo o código-fonte gerado como resultado desta pesquisa foi versionado e encontra-se publicamente disponível no GitHub, através do link: <https://github.com/pedro-ryan/EversonLinks>

CONSIDERAÇÕES

O presente trabalho alcançou com êxito o objetivo de desenvolver uma aplicação *web* independente e de alto desempenho para o gerenciamento e encurtamento de links. A adoção de uma arquitetura *Monorepo* e a utilização de tecnologias modernas, como Bun e ElysiaJS no *back-end*, permitiram a criação de um sistema de redirecionamento extremamente rápido, cumprindo o requisito técnico primordial para ferramentas dessa natureza. O uso do protocolo HTTP 302 demonstrou-se eficaz para evitar o *cache* dos navegadores, garantindo a precisão na contagem de cliques.

No *front-end*, a separação inteligente de estados utilizando o TanStack Query (*React Query*) resultou em uma interface fluida, reativa e livre de recarregamentos desnecessários. Além disso, a implementação de uma interface limpa baseada nos princípios *Developer-First* comprovou que é possível unir alta usabilidade a um *design* sofisticado utilizando bibliotecas de classes utilitárias como o TailwindCSS.

Do ponto de vista acadêmico e profissional, o desenvolvimento deste projeto proporcionou um aprofundamento significativo em práticas de arquitetura de *software* e engenharia *web*. Conclui-se que o sistema "EversonLinks", embora desenvolvido sob o escopo de um Produto Mínimo Viável (MVP), possui uma base arquitetural sólida, escalável e perfeitamente preparada para futuras expansões, como a migração para bancos de dados robustos (PostgreSQL) ou a inclusão de métricas analíticas avançadas.

REFERÊNCIAS BIBLIOGRÁFICAS

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

ELYSIAJS. **Elysia: Ergonomic Framework for Humans**. 2026. Disponível em: <https://elysiajs.com/>. Acesso em: 1 jul. 2026.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a internet: uma abordagem top-down**. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

META. **React: The library for web and native user interfaces**. 2026.

Disponível em: <https://react.dev/>. Acesso em: 1 jul. 2026.

OVEN. **Bun is a fast all-in-one JavaScript runtime**. 2026. Disponível em: <https://bun.sh/>. Acesso em: 1 jul. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

PRISMA DATA. **Prisma: Next-generation Node.js and TypeScript ORM**. 2026. Disponível em: <https://www.prisma.io/>. Acesso em: 1 jul. 2026.

SHADCN. **ui: Beautifully designed components that you can copy and paste into your apps**. 2026. Disponível em: <https://ui.shadcn.com/>. Acesso em: 1 jul. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

TAILWIND LABS. **Tailwind CSS: Rapidly build modern websites without ever leaving your HTML.** 2026. Disponível em: <https://tailwindcss.com/>.

Acesso em: 1 jul. 2026.

TANSTACK. **TanStack Query: Powerful asynchronous state management.** 2026. Disponível em: <https://tanstack.com/query>. Acesso em: 1 jul. 2026.